

ASSIGNMENT

Data Vault 2.0 Design for Company X' s Event Data

1. Overview

Company X's infrastructure produces JSON messages from several backend services (Authorization, Spins, Purchase) that are sent via an internal data bus. The marketing and monetization teams require aggregated reports such as:

- Average purchase per player (across applications)
- The game on which a player spends the most time
- The evolution of average spins over time
- A consolidated user-profile containing Personally Identifiable Information (PII) across all games

The solution must support these analytical requirements while providing historical tracking and auditability. The Data Vault 2.0 methodology is well suited for this purpose because it decouples the raw business keys (Hubs), the relationships (Links), and the descriptive attributes (Satellites), while also supporting agility and scalability.

2. Technology Choices

Data Storage and Processing Platform:

Cloud Data Warehouse:

We recommend using a cloud-based data warehouse such as **Snowflake**, **Amazon Redshift**, or **Google BigQuery**. These platforms offer:

- Scalability and elasticity to handle high volumes of streaming JSON events.
- Native support for semi-structured data formats (such as JSON), which simplifies the initial ingestion.
- Robust SQL support that facilitates transformation, auditing, and reporting.

Data Ingestion/Processing Framework:

- **Apache Kafka or AWS Kinesis** can serve as the message bus for the JSON events.
- An ETL/ELT framework such as **Apache Spark** or a managed solution like **AWS Glue** can be used to parse JSON messages, perform initial quality checks, and load data into the staging area of the data warehouse.

Metadata and Audit:

- A lightweight metadata management tool (or tables within the warehouse) will be used to track load dates, record sources, and hash keys.

3. Data Vault 2.0 Structure

In the Data Vault model, we separate the design into three main types of tables:

1. **Hubs:** Contain unique business keys.
2. **Links:** Define associations between hubs.
3. **Satellites:** Store the descriptive (contextual) data and change history.

3.1. Hubs

3.1.1. Hub_Player

ASSIGNMENT

- **Purpose:** Capture the unique identity of a player (business key: uid from any event).
- **Notes:** Since uid appears in different formats (numeric and string), we standardize it as a string.

Column Name	Data Type	Description
Player_HK	VARCHAR(64)	Surrogate key (hash of the business key).
Player_BK	VARCHAR(100)	Business key - the player' s uid (as string).
Load_Date	TIMESTAMP	Timestamp when the record was loaded.
Record_Source	VARCHAR(50)	Source system indicator (e.g., "auth_event").

3.1.2. Hub_App

- **Purpose:** Capture the unique game/application identifier.
- **Business Key:** app (e.g., "app_3", "app_5").

Column Name	Data Type	Description
App_HK	VARCHAR(64)	Surrogate key (hash of the app business key).
App_BK	VARCHAR(50)	Business key - the app identifier.
Load_Date	TIMESTAMP	Timestamp when the record was loaded.
Record_Source	VARCHAR(50)	Source system indicator.

3.1.3. Hub_Event

- **Purpose:** Capture the unique event message.
- **Business Key:** msg_id (which is unique per message).

Column Name	Data Type	Description
Event_HK	VARCHAR(64)	Surrogate key (hash of the event' s business key).
Event_BK	VARCHAR(50)	Business key - the message id (msg_id).
Event_Type	VARCHAR(20)	The type of event (e.g., "auth_event", "spin_event", "purchase_event").
Publish_TS	TIMESTAMP	The publish timestamp from the event.
Load_Date	TIMESTAMP	Timestamp when the record was loaded.
Record_Source	VARCHAR(50)	Source system indicator.

3.2. Links

3.2.1. Link_Event

- **Purpose:** Link an event to both the player and the application.
- **Notes:**
 - Each event (as defined in Hub_Event) is associated with exactly one player (identified by uid) and one application (identified by app).
 - Although a separate link for each association (Player-Event and App-Event) is possible, a single link table simplifies join paths for the reporting needs.

Column Name	Data Type	Description
Link_HK	VARCHAR(64)	Surrogate key (hash of combined foreign keys).
Event_HK	VARCHAR(64)	Foreign key from Hub_Event.
Player_HK	VARCHAR(64)	Foreign key from Hub_Player.

ASSIGNMENT

Column Name	Data Type	Description
App_HK	VARCHAR(64)	Foreign key from Hub_App.
Load_Date	TIMESTAMP	Timestamp when the record was created in the link.
Record_Source	VARCHAR(50)	Source system indicator.

3.3. Satellites

Since each event type carries specific descriptive attributes, satellites are designed per event type.

3.3.1. Satellite_Auth_Event

- **Purpose:** Store descriptive data for authentication events (PII data).
- **Attributes:** email, phone.

Column Name	Data Type	Description
Event_HK	VARCHAR(64)	Foreign key from Hub_Event (identifies the event).
Email	VARCHAR(255)	Email address of the user.
Phone	VARCHAR(20)	Phone number (stored as a string to accommodate various formats).
Load_Date	TIMESTAMP	Timestamp when the satellite record was loaded.
Record_Source	VARCHAR(50)	Source system indicator.

3.3.2. Satellite_Spin_Event

- **Purpose:** Store descriptive data for spin events.
- **Attributes:** spin (the spin value).

Column Name	Data Type	Description
Event_HK	VARCHAR(64)	Foreign key from Hub_Event.
Spin	INTEGER	Spin value.
Load_Date	TIMESTAMP	Timestamp when the record was loaded.
Record_Source	VARCHAR(50)	Source system indicator.

3.3.3. Satellite_Purchase_Event

- **Purpose:** Store descriptive data for purchase events.
- **Attributes:** amount (purchase amount).

Column Name	Data Type	Description
Event_HK	VARCHAR(64)	Foreign key from Hub_Event.
Amount	INTEGER	Purchase amount (in minor currency units, e.g., cents).
Load_Date	TIMESTAMP	Timestamp when the record was loaded.
Record_Source	VARCHAR(50)	Source system indicator.

4. Data Flow and Additional Components

To support this Data Vault model, several additional components and processes need to be developed:

4.1. Ingestion and ETL/ELT Pipeline

ASSIGNMENT

Message Capture and Staging:

- **Ingestion Layer:** Use Kafka/Kinesis to capture JSON messages from the internal data bus.
- **Staging Area:** Load raw JSON data into a staging area in the data warehouse. Use native support for JSON (or a blob store) to temporarily persist raw events.

Parsing and Transformation:

- **ETL/ELT Jobs:** Using Apache Spark, AWS Glue, or similar, parse the JSON messages. During the transformation:
 - **Standardize Formats:** Convert numeric and string `uid` values into a consistent string format.
 - **Hash Keys Generation:** Generate surrogate keys (hash values) for hubs and link tables.
 - **Data Quality Checks:** Validate required fields and filter or log any records with missing business keys.
- **Loading Data into Data Vault:** Insert records into the appropriate Hub, Link, and Satellite tables.

4.2. Metadata and Audit Tracking

- **Metadata Repository:**
 - Create tables or use built-in features of the chosen data warehouse to track load dates, record sources, and any ETL errors.
 - Implement audit tables to track changes to the satellites for historical analysis.

4.3. Security and Data Governance

- **PII Handling:**
 - Ensure that PII (e.g., email and phone from authentication events) is stored in a secure manner.
 - Implement access controls at the data warehouse level to restrict PII access only to authorized users.

4.4. Data Consumption

- **Reporting and Analytics:**
 - Develop views or data marts that join hubs, links, and satellites for simplified reporting (e.g., average purchase per player, game usage patterns, and spin trends).
 - Optionally, integrate BI tools (e.g., Tableau, Power BI) that connect directly to the data warehouse.

5. Summary

In this design, we use a cloud data warehouse (e.g., Snowflake, Amazon Redshift, or BigQuery) as our storage solution with an ingestion pipeline built around Kafka/Kinesis and Apache Spark or AWS Glue. Our Data Vault 2.0 model consists of:

Hubs:

- **Hub_Player:** Stores unique player identifiers (`uid`).
- **Hub_App:** Stores unique game/application identifiers.
- **Hub_Event:** Stores unique event messages (`msg_id`) along with the event type and publish timestamp.

ASSIGNMENT

Link:

- **Link_Event:** Connects each event to its corresponding player and application.

Satellites:

- **Satellite_Auth_Event:** Stores authentication event details (PII – email, phone).
- **Satellite_Spin_Event:** Stores spin event details (spin value).
- **Satellite_Purchase_Event:** Stores purchase event details (amount).

Additional components include robust ETL/ELT pipelines, metadata/audit tracking, and security measures for sensitive data.

This modular design not only meets the reporting requirements but also scales to accommodate future event types and business needs, following the agile principles of Data Vault 2.0.